

There are a few noteworthy points here.

- As specified in the prop validation criteria, the tweet-component expects a tweet object prop (**props: {tweet: Object}**). Vue can emit alerts if the component is made with a tweet prop that is not an object.
- Using the Mustache syntax:{{}}, we connect the properties of the tweet object prop to the component template.
- Bulma's Box element is adapted in the part markup since it bears a striking similarity to a tweet.

In the HTML template, we'll need to build the markup for the location of our Vue application (i.e., the element with the **id** of the app). We'll use the **v-for** directive to collect tweets within this markup. Given that we'll be iterating over tweets, the tweet is an acceptable alias to use in the directive. Additionally, we'll move the iterated tweet object as props to each made tweet component so that it can be accessed inside the component.

```
<div id="app" class="columns">
  <div class="column">
    <tweet-component v-for="tweet in tweets" :tweet="tweet"/>
  </div>
</div>
```

Copyright: CSS tricks

HTML Template for using tweets

Maintaining State

By default, as Vue updates a collection of items made with **v-for**, it employs an “in-place patch” technique. If the order of the data items changes, Vue may patch each feature in place to ensure it accurately represents what should be made at that specific index.

Although this default mode is effective, it is only appropriate when your list render performance does not depend on the child component or temporary DOM state (e.g., form input values).

To provide Vue with a clue that it can trace the identity of each node, and therefore reuse and reorder existing items, you must provide each object with a specific main attribute:

```
<div v-for="item in items" v-bind:key="item.id">
  <!-- content -->
</div>
```

Copyright: CSS tricks

Applying key attribute